SqueezeRadar

(Squeezy)

Open-Source Real-Time Behavioral Finance Intelligence Platform

A structured technical project document describing the system architecture, data collection pipeline, NLP/ML scoring workflow, repository organization, local setup, and research-use policy.

Project Scope: Non-commercial, read-only research and personal data analysis.

MIT License • Python 3.10+ • FastAPI • React • Kafka • MongoDB • TimescaleDB • Redis • Cassandra

1. Project Overview

SqueezeRadar (Squeezy) is an open-source, real-time behavioral finance intelligence platform designed to detect emerging market microstructure anomalies and short-squeeze conditions. The system fuses alternative social data streams with market signals across 400+ monitored US equities.

Research-use boundary: SqueezeRadar is designed exclusively as a non-commercial, read-only research tool for personal data analysis. It does not perform automated posting, commenting, voting, messaging, or other user-account actions on social platforms.

2. Reddit API & Responsible Builder Policy Compliance

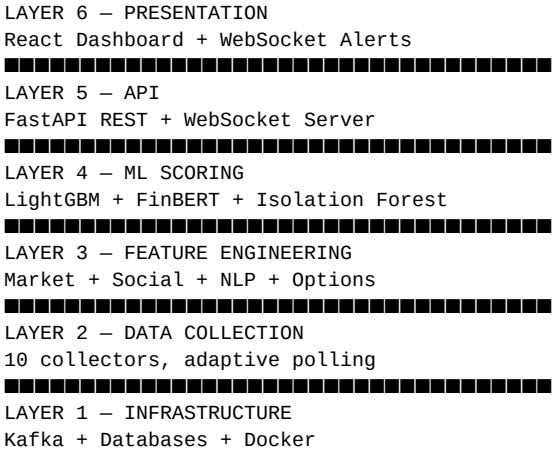
SqueezeRadar ingests public discussion data through Reddit's Data API. The project documentation specifies read-only collection and describes controls intended to follow Reddit's Developer Terms and Responsible Builder Policy.

Area	Implementation
Read-Only Operations	The reddit_collector passively fetches public submission titles, selftext, and top-level comment metadata. It contains no capability or write scopes for posting, commenting, upvoting, or direct messaging.
Rate-Limit Adherence	API calls are routed through custom request throttling that monitors X-Ratelimit-Remaining and X-Ratelimit-Reset response headers. The stated limit is no more than 30 requests per minute per developer key.
User-Agent	Requests use: script:com.squeezeradar.app:v1.0 (by /u/PieceOk9028)
Data Retention & Privacy	Raw social posts are retained for short-term sentiment scoring with a stated 90-day TTL index in MongoDB and are not intended for sharing, resale, or commercial redistribution.

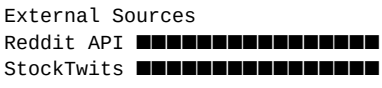
Why Devvit Is Not Applicable

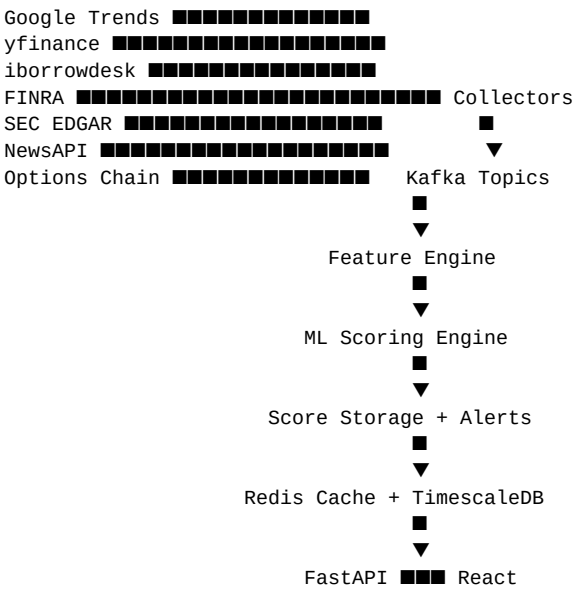
The repository is described as a standalone Python 3.10+ data-science and machine-learning pipeline using FinBERT, KeyBERT, BERTopic, LightGBM, and a multi-database infrastructure involving TimescaleDB, MongoDB, Redis, and Cassandra. The project therefore uses its own external analytics architecture rather than relying on a Reddit-hosted application architecture.

3. System Architecture



High-Level Data Flow





4. Data Collection & Subreddit Scope

The pipeline targets public retail discussion within the following financial research communities:

• r/wallstreetbets

• r/stocks

• r/investing

• r/options

Adaptive Polling

Polling is described as rate-throttled and dynamically scaled according to ticker state:

Ticker State	Polling Range
Early Setup / Cooldown	10–15 minutes
Heating Up / Active	1–5 minutes
High Activity Spike	30 seconds (hard-capped by rate-limit tokens)

Collector Inventory

Collector	Data Source	Refresh Rate	Target Storage
reddit_collector	Reddit API (PRAW)	Adaptive (30s–15min)	MongoDB
stocktwits_collector	StockTwits API	30 minutes	MongoDB
pytrends_collector	Google Trends	Hourly	PostgreSQL
yfinance_collector	Yahoo Finance	15 min (Market Hours)	TimescaleDB
iborrowdesk_collector	iborrowdesk.com	3× daily	Cassandra
finra_collector	FINRA Short Sale Files	Daily (Next day)	Cassandra
sec_edgar_collector	SEC EDGAR API	Daily	PostgreSQL
options_collector	Options Chain Feed	15 minutes	PostgreSQL
news_collector	Financial RSS / News	15 minutes	MongoDB

5. NLP & Machine Learning Scoring Pipeline

1. FinBERT Analysis

Evaluates financial sentiment as bullish, bearish, or neutral across collected discussion content.

2. KeyBERT & BERTopic

Identifies keyword urgency and groups emerging ticker discussion clusters.

3. Feature Vector Assembly

Combines social metrics such as velocity, volume, and diversity with market microstructure metrics such as short interest, borrow rate, and volume acceleration into a 28-feature vector.

4. Ensemble ML Models

Combines lifecycle classification, anomaly detection, and historical-condition similarity models.

Model	Role
Lifecycle Classifier (LightGBM)	Predicts squeeze phase transitions.
Hype Anomaly Detector (Isolation Forest)	Flags abnormal social-volume spikes.
Outcome Similarity Predictor (XGBoost)	Measures similarity to historical liquidity setups.

6. Repository Structure

```
Squeezy/
■■■ config/                 # Environment and pipeline settings
■■■ collectors/             # Data ingestion modules
■   ■■■ reddit_collector.py  # PRAW Reddit API reader (Read-Only)
■   ■■■ market_collector.py  # Price/volume collectors
■   ■■■ options_collector.py # Derivatives feed
■■■ nlp/                    # FinBERT & sentiment analysis modules
■■■ ml/                     # LightGBM scoring & feature engineering
■■■ api/                    # FastAPI backend endpoints
■■■ dashboard/              # React frontend dashboard
■■■ docker-compose.yml      # Local container orchestration
■■■ requirements.txt        # Python dependencies
■■■ README.md
```

7. Getting Started

Prerequisites

• Python 3.10+

• Docker & Docker Compose

• Reddit Developer API credentials (client_id, client_secret)

Installation & Local Setup

1. Clone the Repository

```
git clone https://github.com/harshith-shiva/Squeezy.git
cd Squeezy
```

2. Configure Environment Variables

```
# Reddit API Credentials (Read-Only)
REDDIT_CLIENT_ID="your_client_id"
REDDIT_CLIENT_SECRET="your_client_secret"
REDDIT_USER_AGENT="script:com.squeezeradar.app:v1.0 (by /u/PieceOk9028)"
```

```
# Database Settings
POSTGRES_DB=squeezeradar
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres_password
MONGO_URI=mongodb://localhost:27017/
REDIS_HOST=localhost
REDIS_PORT=6379
```

3. Start Infrastructure Services

```
docker-compose up -d
```

4. Install Python Dependencies

```
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

5. Run the API & Collectors

```
# Run the Reddit ingestion worker
python -m collectors.reddit_collector

# Start the FastAPI server
uvicorn api.main:app --reload
```

8. License & Research Disclaimer

License: Distributed under the MIT License. See the LICENSE file in the repository.

> Research Disclaimer: SqueezeRadar is developed solely for educational, academic, and personal analytical research. It does not provide financial advice, trading signals, or investment recommendations.

Document Notes

This document is a structured presentation of the project specification supplied for SqueezeRadar (Squeezy). External platform terms, API limits, retention requirements, and service availability can change over time; maintainers should verify current third-party policies before deployment.

End of Document